



به نام خدا

آزمایشگاه سیستم عامل - بهار ۱۴۰۵



پروژه سوم: زمان‌بندی در xv6

طراحان: [البرز محمودیان](#) - [کسری کاشانی](#)

اهداف پروژه

در این پروژه به طور کلی با مفهوم زمان‌بندی^۱ و به شکل عملی در سیستم عامل xv6 آشنا خواهید شد. در این راستا، ابتدا الگوریتم زمان‌بندی سیستم عامل xv6 بررسی خواهد شد. سپس به پیاده‌سازی یک زمان‌بند چند لایه^۲ می‌پردازید. در نهایت با استفاده از فراخوانی‌های سیستمی و برنامه‌های سطح کاربر، از صحت پیاده‌سازی خود اطمینان حاصل خواهید کرد.

مقدمه

یکی از مهم‌ترین وظایف سیستم عامل‌ها، تخصیص منابع سخت‌افزاری به برنامه‌های سطح کاربر است. در این امر، پردازنده که یکی از منابع فعال^۳ می‌باشد، نیازمند یک زمان‌بند مناسب به منظور اجرا شدن هر چه بهتر پردازنده‌ها می‌باشد. از آنجایی که زمان‌بند، نیازمند دانستن اطلاعات سیستم و همچنین وضعیت هر کدام از پردازنده‌ها^۴ می‌باشد، غالباً در سطح کرنل اجرا می‌شود که در xv6 نیز همین گونه است.

xv6 سیستم عاملی است که از مدل چند پردازشی متقارن^۵ پشتیبانی می‌کند. با این قابلیت، هر پردازنده، یک کپی از کد اجرایی زمان‌بند (تابع زمان‌بند) و همچنین متن^۶ زمان‌بند خود را دارد و پس از انتخاب پردازنده

^۱ Scheduling

^۲ Multi-Level Scheduler

^۳ Active

^۴ Processes

^۵ Symmetric Multiprocessing (SMP)

^۶ Context

از صف پردازه‌های آماده به اجرا^۷، توسط زمان‌بند (طبق الگوریتم زمان‌بندی)، با انجام عملیات تعویض متن^۸ از متن زمان‌بند به متن پردازه، اجرا خواهد شد.

یکی از ساده‌ترین و در عین حال کاربردی‌ترین الگوریتم‌های زمان‌بندی، زمان‌بندی نوبت‌گردشی^۹ است که xv6 نیز از همین الگوریتم استفاده می‌کند. در سیستم‌های امروزه اما، به دلیل رشد نیاز به نشان دادن واکنش مناسب به اتفاقات متنوع در سیستم، نیاز به استفاده از الگوریتم‌های زمان‌بندی پیشرفته‌تر نیز رشد کرده است. در جدول زیر، چند مورد از الگوریتم‌های زمان‌بندی مورد استفاده در سیستم عامل‌های مختلف آورده شده است:

سیستم عامل	الگوریتم زمان‌بندی	توضیحات
Windows NT/Vista/7	MLFQ ^{۱۰}	دارای ۳۲ سطح اولویت و ۲ صف است. اولویت‌های: • تا ۱۵ برای تسک‌های عادی و • ۱۶ تا ۳۱ برای تسک‌های بی‌درنگ ^{۱۱} .
Mac OS X	MLFQ	دارای ۱۲۷ سطح اولویت و ۴ صف اصلی است. اولویت‌های: • تا ۵۱ تسک‌های عادی، • ۵۲ تا ۷۹ تسک‌های با اهمیت‌تر، • ۸۰ تا ۹۵ رزرو شده برای ریسسه‌های کرنل ^{۱۲} و • ۹۶ تا ۱۲۷ ریسسه‌هایی که اولویتشان بر مبنای نسبتی از سیکل‌های کلاک پردازنده که به آن‌ها تعلق می‌گیرد، تعریف می‌شود.
FreeBSD/NetBSD	MLFQ	دارای ۲۵۵ سطح اولویت و ۵ کلاس اولویت‌بندی اصلی است.
Solaris	MLFQ	دارای ۵ کلاس که بازه اولویت‌بندی برای هر کلاس متفاوت است.
2.5 < Linux < 2.6.23	O(1)	مرتبه اجرای ثابت.

^۷ Ready Queue

^۸ Context Switch

^۹ Round Robin

^{۱۰} Multilevel Feedback Queue (MFQ)

^{۱۱} Real-time

^{۱۲} Kernel Threads

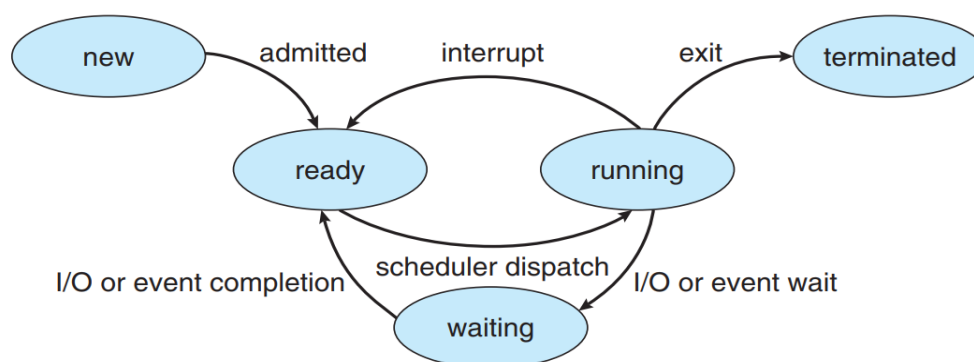
به هر پردازش، بر اساس سطح اولویت و میزان استفاده، زمان مناسبی برای پردازنده اختصاص دهد و اولویت‌های ثابت در آن کمتر دخیل هستند.	CFS ¹³	Linux > 2.6.23
همه فرآیندها از اولویت یکسانی برخوردارند و به ترتیب نوبتی اجرا می‌شوند، که باعث می‌شود زمان‌بندی مناسب برای سیستم‌های ساده و بی‌درنگ باشد، اما در مدیریت پیچیده‌تر اولویت‌ها و تسک‌ها محدودیت دارد.	RR	xv6

همانطور که مشاهده می‌کنید امروزه، زمان‌بندی از نوع MLFQ بسیار محبوب است. در این تمرین نیز با شکلی از این زمان‌بندی آشنا خواهید شد.

شرح پروژه

قبل از پیاده‌سازی زمان‌بند مورد نظر در این تمرین، ابتدا نیاز است تا با مفاهیم اولیه و همچنین نمود آن‌ها در xv6 آشنایی داشته باشید. در این قسمت به همین موضوع خواهیم پرداخت.

به طور کلی هر پردازش در چرخه حیات¹⁴ خود شامل وضعیت‌های مختلفی است. شکل ۱ که مربوط به فصل ۳ منبع درس می‌باشد، این چرخه را به خوبی توصیف کرده است.



شکل ۱. چرخه وضعیت یک پردازش.

¹³ Completely Fair Scheduler

¹⁴ Life Cycle

xv6 نیز از این قاعده مستثنا نیست و به منظور زمان‌بندی و مدیریت پردازش‌ها، وضعیت هر کدام از پردازش‌ها را در بلوک کنترل پردازش¹⁵ آن پردازش نگه می‌دارد. عمده عملیات زمان‌بندی و اقدامات مربوط به آن و همچنین داده‌ساختارهای مرتبط، در فایل‌های proc.c و proc.h قابل مشاهده هستند.

۱) ساختار PCB و همچنین وضعیت‌های تعریف شده برای هر پردازش را در xv6 پیدا کرده و گزارش کنید. آیا شباهتی میان داده‌های موجود در این ساختار و ساختار به تصویر کشیده شده در شکل 3.3 منبع درس وجود دارد؟ (ذکر حداقل ۵ مورد و معادل آن‌ها در xv6)

۲) هر کدام از وضعیت‌های تعریف شده معادل کدام وضعیت در شکل ۱ می‌باشند؟

همان‌طور که در شکل ۱ نیز مشاهده می‌کنید، هر پردازش در هر وضعیتی که باشد، توسط رویدادهای مختلف به وضعیت‌های دیگر گذار¹⁶ می‌کند.

تغییر وضعیت در xv6

در این قسمت با گذارهای نمایش داده شده در شکل ۱ و معادل آن‌ها در xv6 آشنا می‌شوید.

Admitted

پس از اجرای سیستم عامل، اولین پردازش (initcode) توسط تابع userinit ساخته خواهد شد (main:36). در این تابع، مقداردهی‌های اولیه انجام می‌شود تا پردازش مربوط به برنامه سطح کاربر init.c ایجاد شود. در این برنامه، اولین پردازش (initproc)، عملیات fork را انجام می‌دهد تا پردازش جدیدی تحت رابطهٔ فرزند با initproc ایجاد شود (init.c:24). این پردازش جدید در واقع بستر اجرای بقیه برنامه‌های سطح کاربر در xv6 یا همان shell می‌باشد که با آن کم و بیش در آزمایشگاه اول آشنا شدید. در واقع تمامی برنامه‌های سطح کاربر fork و exec ای از پردازش sh یا همان shell می‌باشند.

۳) با توجه به توضیحات گفته شده، کدام یک از توابع موجود در proc.c منجر به انجام گذار از حالت new به حالت ready که در شکل ۱ به تصویر کشیده شده، خواهد شد؟ وضعیت یک پردازش در xv6 در این گذار از چه حالتی/حالت‌هایی به چه حالت/حالت‌هایی تغییر می‌کند؟ پاسخ خود را با پاسخ سوال ۲ مقایسه کنید.

¹⁵ Process Control Block (PCB)

¹⁶ Transition

Scheduler Dispatch

با قرار گرفتن یک پردازش در حالت ready، پس از گرفتن نوبت توسط زمان‌بند، پردازش اجرا خواهد شد. تابع scheduler وظیفه این نوبت‌دهی را بر عهده دارد (proc.c:323).

همانطور که پیش‌تر اشاره شد، هر پردازنده در xv6، یک اجرای مستقل از زمان‌بند خود را دارد. اما نکته حائز اهمیت این است که در xv6 تمامی اطلاعات پردازش‌های موجود، در یک جدول مشترک قرار دارند (proc.c:10). به منظور جلوگیری از رقابت¹⁷ میان دو پردازنده در این جدول و ایجاد تغییرات همزمان، صف پردازش‌ها با استفاده از یک قفل محافظت شده است. قفل و مکانیزم قفل‌گذاری در ادامه درس و همچنین آزمایشگاه بعد بررسی خواهد شد.

۴) سقف تعداد پردازش‌های ممکن در xv6 چه عددی است؟ در صورتی که یک پردازش تعداد زیادی پردازش فرزند ایجاد کند و از این سقف عبور کند، کرنل چه واکنشی نشان داده و برنامه سطح کاربر چه بازخوردی دریافت می‌کند؟

تابع scheduler در یک حلقه بی‌نهایت اجرا می‌شود و الگوریتم زمان‌بندی را اجرا می‌کند. ابتدا وقفه‌ها را فعال می‌کند و سپس با قفل کردن ptable، پردازش واجد شرایط اجرا شدن (RUNNABLE) را طبق الگوریتم پیدا می‌کند.

پس از انتخاب پردازش توسط زمان‌بند، به پردازنده‌ای که زمان‌بند آن فعال شده است واگذار و سپس عملیات تعویض متن انجام می‌شود و وضعیت پردازش به حالت RUNNING تغییر پیدا می‌کند. با پایان یافتن اجرای پردازش، سیستم به فضای کرنل برمی‌گردد و از ادامه جایی که ابتدا تعویض متن رخ داده بود، اجرای زمان‌بند آن هسته، ادامه پیدا می‌کند (proc.c:346,347). پس، تابع scheduler یک بار تا انتهای صف می‌رود و هر پردازش را به شیوه‌ای که گفته شد اجرا می‌کند. بعد از بررسی آخرین پردازش در صف، قفل جدول پردازش‌ها را آزاد کرده و سپس حلقه بی‌نهایت تکرار خواهد شد و این فرایند از ابتدای صف مجدداً انجام می‌شود.

۵) چرا نیاز است در ابتدای هر حلقه تابع scheduler، جدول پردازش‌ها قفل شود؟ آیا در سیستم‌هایی که تنها یک پردازنده (یک هسته پردازشی) دارند هم نیاز است این کار صورت بگیرد؟

¹⁷ Race/Contention

Context Switch

تعویض متن به فرایندی گفته می‌شود که در آن سیستم عامل یک پردازش در حال اجرا را متوقف کرده و یک پردازش جدید را اجرا می‌کند. همچنین در صورت وقوع یک وقفه¹⁸ مثل وقفه تایمر یا یک وقفه از سمت سخت‌افزار، سیستم عامل نیاز دارد تا پردازش جاری را متوقف کرده و به وقفه رسیدگی کند.

در xv6، تعویض متن با استفاده از تابع switch انجام می‌گیرد که وظیفه ذخیره و بازیابی وضعیت را دارد:

```
void swtch(struct context *old, struct context *new);
```

این تابع وضعیت پردازش جاری را در old ذخیره می‌کند و وضعیت پردازش جدید را از new بارگذاری می‌کند. ساختار context وظیفه نگهداری آخرین وضعیت پردازش را دارد.

۶) همانطور که می‌دانید یکی از مهم‌ترین رجیسترها قبل از هر تعویض متن Program Counter است که نشان می‌دهد روند اجرای برنامه تا کجا پیش رفته است. با ذخیره‌سازی این رجیستر می‌توان محل ادامه برنامه را بازیابی کرد. این رجیستر در ساختار context چه نام دارد؟ این رجیستر چگونه قبل از انجام تعویض متن ذخیره می‌شود؟

Interrupt

تابع trap (trap.c:37) در سیستم‌عامل xv6 برای مدیریت وقفه‌ها به کار می‌رود. در صورتی که پردازش در حالت RUNNING باشد و وقفه تایمر (یعنی T_IRQ0+IRQ_TIMER) ایجاد شود، منجر به دو رویداد می‌شود. ابتدا زمان سیستم به اندازه یک واحد افزایش پیدا می‌کند؛ که یکی از هسته‌های پردازنده مسئولیت این موضوع را بر عهده دارد (trap:50). رویداد دوم نیز در صورتی که پردازش‌ای در حالت اجرا باشد، تابع yield برای آن فراخوانی می‌شود (trap:106).

۷) همانطور که در قسمت قبل مشاهده کردید، ابتدای تابع scheduler، ایجاد وقفه به کمک تابع sti فعال می‌شود. با توجه به توضیحات این قسمت، اگر وقفه‌ها فعال نمی‌شد، چه مشکلی به وجود می‌آمد؟

۸) به نظر شما وقفه تایمر هر چند مدت یک بار صادر می‌شود؟ (راهنمایی: می‌توانید با اضافه کردن یک cprintf پس از ++ticks این موضوع را مشاهده کنید).

¹⁸ Interrupt

تابع yield (proc.c:386) وظیفه تغییر حالت پردازش از RUNNING به RUNNABLE را دارد؛ یعنی پردازش از اجرا خارج شده و پردازنده را رها می‌کند تا زمان‌بند در خصوص اجرای پردازش‌ها مجدداً تصمیم بگیرد. فرایند آن نیز به این شکل است:

ابتدا با قفل کردن جدول پردازش‌ها¹⁹، پردازش فعلی در جدول به حالت RUNNABLE تغییر وضعیت می‌دهد. سپس تابع sched فراخوانی می‌شود تا زمان‌بند، امکان انتخاب پردازش بعدی را داشته باشد. در پایان، قفل جدول پردازش‌ها آزاد می‌شود و کنترل به زمان‌بند برمی‌گردد. در نهایت تابع sched (proc.c:366)، یک تعویض متن از پردازش کنونی به تابع scheduler مربوط به همین پردازنده انجام می‌دهد.

۹) با توجه به توضیحات داده شده، چه تابعی منجر به انجام شدن گذار interrupt در شکل ۱ خواهد شد؟

۱۰) با توجه به توضیحات قسمت scheduler dispatch می‌دانیم زمان‌بندی در xv6 به شکل نوبت گردشی است. حال با توجه مشاهدات خود در این قسمت، استدلال کنید، کوانتوم زمانی این پیاده‌سازی از زمان‌بندی نوبت گردشی چند میلی ثانیه است؟

Wait

در صورتی که پردازش نیازمند به وقوع پیوستن اتفاقی برای ادامه انجام پردازش خود باشد، مانند عملیات I/O یا منتظر ماندن برای یک رویداد، برای آن صبر می‌کند و به حالت waiting می‌رود. در این حالت، پس از به وقوع پیوستن اتفاق مورد نظر، سیستم عامل وظیفه دارد تا پردازش را از این اتفاق آگاه سازد.

یکی از این رویدادها، منتظر ماندن برای اتمام پردازش فرزند از طرف والد می‌باشد که به کمک تابع wait انجام می‌شود. این تابع به والد این امکان را می‌دهد تا منتظر خاتمه کار یکی از فرزندان خود باشد و زمانی که یکی از آن‌ها به پایان رسید، والد را از اتمام آن آگاه و از سیستم پاک‌سازی کند.

۱۱) تابع wait در نهایت از چه تابعی برای منتظر ماندن برای اتمام کار یک پردازش استفاده می‌کند؟

۱۲) با توجه به پاسخ سوال قبل، استفاده(های) دیگر این تابع چیست؟ (ذکر یک نمونه)

۱۳) با توجه به پاسخ سوال ۷، این تابع منجر به گذار از چه وضعیتی به چه وضعیتی در شکل ۱ خواهد شد؟

¹⁹ Process Table (ptable)

Exit

در انواع حالات پردازها، حالت ZOMBIE نشان‌دهندهٔ پردازهای است که اجرای آن به پایان رسیده، اما هنوز منابع آن به‌طور کامل آزاد نشده و منتظر است تا والد از طریق wait، آن را جمع‌آوری کند. با فراخوانی تابع exit در xv6، وضعیت پرداز به حالت ZOMBIE می‌رود و والد پرداز به حالت RUNNABLE تغییر می‌کند تا در فرصت بعدی که اجرا شد، فرزند خاتمه‌یافتهٔ خود را جمع‌آوری کند.

(۱۴) در بخش 3.3.2 منبع درس، با پردازهای Orphan آشنا شدید. رویکرد xv6 در رابطه با این گونه پردازها چیست؟

زمان‌بندی بازخوردی چند سطحی

همان‌طور که پیش‌تر اشاره شد، امروزه نیازمند استفاده از الگوریتم‌های زمان‌بندی پیشرفته‌تر هستیم. یکی از این الگوریتم‌های پرکاربرد، زمان‌بندی بازخوردی چند سطحی می‌باشد که در درس با آن آشنا شده‌اید. در این قسمت قصد داریم پیاده‌سازی‌ای از این الگوریتم در xv6 داشته باشیم.

سطح اول: زمان‌بند نوبت گردشی با کوانتوم زمانی

برای شروع، ابتدا زمان‌بندی xv6 را به حالتی تغییر دهید که کوانتوم زمانی نوبت گردشی آن ۵۰ میلی ثانیه (بر حسب تیک زمانی سیستم) باشد. برای اطمینان از صحت پیاده‌سازی خود، xv6 را در حالتی اجرا کنید که تعداد پردازنده‌های مورد استفاده ۱ عدد باشد. برای این کار می‌توانید در هنگام زدن دستور make، پرچم $CPUS^{20}$ را برابر با ۱ قرار دهید و یا در Makefile این متغیر را تغییر دهید.

در قدم بعدی، در فایل trap.c از cprintf استفاده کنید و pid پرداز در هر تیک از اجرا و یا pid پرداز همراه با مدت زمان اجرای آن بر حسب تیک زمانی سیستم را چاپ کنید. در نهایت، برنامهٔ سطح کاربری‌ای بنویسید که درون خود چند پرداز ایجاد کند و هر پرداز مشغول انجام عملیاتی شود که به اندازه کافی طول می‌کشد. تصویری از خروجی اجرای سیستم عامل در این حالت را در گزارش خود قرار دهید.

²⁰ Flag

راهنمایی: به دلیل وجود صف‌های دیگر در زمان‌بندی و اجرای آن‌ها، احتمالاً نیاز داشته باشید تا اندیس آخرین پردازش اجرا شده برای صف نوبت‌گردشی را نگه دارید تا در دفعه بعد، از ادامه صف، زمان‌بندی را از پی بگیرید. (به دلیل نیاز به تغییر شرایطی که در قسمت Scheduler Dispatch توضیح داده شد).

۱۵) مقدار CPU را مجدداً به عدد ۲ برگردانید. آیا همچنان ترتیبی که قبلاً مشاهده می‌کردید پا برجاست؟ علت این امر چیست؟

برای صحت درستی اجرای دیگر الگوریتم‌های زمان‌بندی نیز می‌توانید از همین روش استفاده کنید و سپس هر ۳ سطح را در xv6 کنار هم قرار داده و صحت‌سنجی نهایی را انجام دهید.

سطح دوم: اول، کوتاه‌ترین کار²¹

در درس با زمان‌بند SJF آشنا شدید. یکی از مشکلات این زمان‌بند، عدم امکان پیاده‌سازی به دلیل Halting Problem است. در این قسمت قصد داریم تا با استفاده از روش‌های تقریبی، یک پیاده‌سازی از این زمان‌بند داشته باشیم که از همان ایده‌ی SJF استفاده می‌کند. لذا این زمان‌بند در حقیقت پیاده‌سازی دقیقی از SJF واقعی نیست.

در این زمان‌بند، هر پردازش دارای یک عدد پیشنهادی در مورد زمان اجرا²² و نیز یک عدد دیگر بیانگر سطح اطمینان²³ از زمان اجرا می‌باشد. حال هر بار عددی بین ۰ تا ۹۹ (باقی‌مانده به ۱۰۰) به طور تصادفی انتخاب می‌شود و در صورتی که این عدد تولید شده کوچکتر یا مساوی سطح اطمینان بود، آن پردازش برای اجرا انتخاب می‌شود و در غیر این صورت، پردازش بعدی طبق سیاست SJF انتخاب می‌شود.

با یک مثال، عملکرد این زمان‌بند را روی پردازش‌های جدول زیر بررسی خواهیم کرد:

Process	Burst Time	Confidence
p1	10	80
p2	8	60
p3	12	90

²¹ Shortest Job First (SJF)

²² Burst Time

²³ Confidence

بر اساس الگوریتم SJF، ابتدا پردازهای که کمترین زمان اجرا را دارد به عنوان کاندیدا انتخاب خواهد شد؛ که در اینجا پردازهی p2 می‌باشد. سپس عددی به طور تصادفی انتخاب می‌شود و اگر کوچکتر یا مساوی سطح اطمینان این پردازه یعنی 60 بود، همین پردازه اجرا می‌شود و در غیر این صورت، پردازهی بعدی با کمترین زمان اجرا یعنی p1 انتخاب می‌شود. سپس دوباره عددی انتخاب می‌شود و اگر از سطح اطمینان این پردازه یعنی 80 بیشتر بود، دوباره به سراغ پردازهی بعدی می‌رویم. اما از آنجا که پردازهی بعدی یعنی p3 پردازهی آخر می‌باشد، مستقل از سطح اطمینانش، آن را اجرا خواهیم کرد.

نکته: مقادیر پیش‌فرض برای زمان اجرا و سطح اطمینان را به ترتیب برابر 2 و 50 در نظر بگیرید.

سطح سوم: اولین ورود، اولین رسیدگی²⁴

با الگوریتم FCFS نیز در درس آشنا شده‌اید. در این زمان‌بندی، اولین پردازهای که وارد صف شده باشد، سریع‌تر انتخاب می‌شود و الگوریتم تا اتمام کامل این پردازه، سراغ پردازهی دیگری نخواهد رفت.

راهنمایی: برای پیاده‌سازی این الگوریتم، نیاز است زمان (tick) وارد شدن پردازه به این صف را ذخیره کنید و با استفاده از آن، پردازهای که زودتر وارد صف شده را پیدا کرده و اولویت انتخاب توسط زمان‌بند را به آن اختصاص دهید. دقت کنید که الگوریتم FCFS بر اساس آخرین زمان ورود پردازه به این صف انجام می‌شود، و نه بر اساس زمان ایجاد پردازه.

برش‌دهی زمانی²⁵

در حالت کلی، می‌توان ارتباطات میان صف‌ها را به دو صورت Fixed Priority یا time-slicing (در بعضی کاربردها time-sharing) تعیین کرد. در حالت Fixed Priority، اولویت صف‌ها ثابت هستند و اولویت اجرا همواره با صفی است که دارای بیشترین اولویت است و تنها در صورت نبود پردازهی آماده به اجرا در این صف، به صف با اولویت پایین‌تر مراجعه خواهیم کرد. اما در روش time-slicing، به هر صف قسمتی از زمان پردازنده برای اجرا را اختصاص می‌دهیم و پردازها طی این زمان، با سیاست حاکم بر داخل صف اجرا خواهند شد.

²⁴ First Come First Served (FCFS)

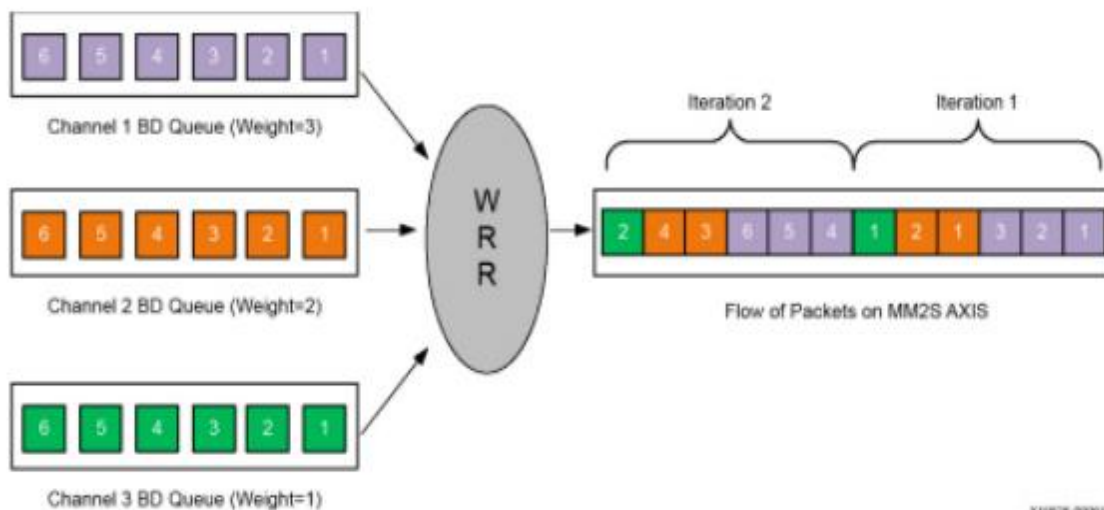
²⁵ Time Slicing

در این تمرین قصد داریم به جای استفاده از اولویت ثابت میان صف‌ها، از روش time-slicing استفاده کنیم تا به دیگر صف‌ها نیز فرصت اجرا داده شود. حال این روش را با استفاده از سیاست نوبت گردشی وزن دار²⁶ پیاده‌سازی می‌کنیم. برای این کار در نظر بگیرید که برش زمانی واحد (Weight=1)، برابر با ۱۰۰ میلی‌ثانیه است و وزن سطح‌های اول، دوم و سوم نیز به ترتیب برابر با ۳، ۲ و ۱ می‌باشند.

در صورتی که در یک صف پردازهای وجود نداشته یا پردازش‌های موجود در یک صف زودتر از زمانی که سیاست به آن‌ها تخصیص داده بود، تمام شوند، صف بعدی اجرا خواهد شد ولی برش زمانی باقی‌مانده از صف قبلی به صف بعدی منتقل نخواهد شد و صف بعدی طبق برش زمانی پیش‌فرض خود اجرا خواهد شد.

نکته: همانطور که پیش‌تر اشاره شد، هر هسته دارای زمانبند خود می‌باشد. در نتیجه در این نحوه‌ی زمان‌بندی می‌بایست هر CPU با توجه به برش زمانی خود زمان‌بندی را انجام دهد. برای این کار می‌توانید با اضافه کردن داده‌های مورد نیاز خود به ساختار cpu در فایل proc.h، این داده‌ها را در توابع scheduler و trap تغییر داده و بر اساس آن‌ها تصمیم‌گیری لازم را انجام دهید.

برای نحوه‌ی کارکرد این سیاست می‌توانید از شکل ۲ نیز کمک بگیرید.



شکل ۲. نحوه عملکرد نوبت گردشی وزن دار.

²⁶ Weighted Round Robin (WRR)

(۱۶) با توجه به سیاست‌های پیاده‌سازی شده در سطوح دوم و سوم و همچنین استفاده از روش -time slicing، چرا همچنان مشکل starvation امکان رخ دادن دارد؟

سازوکار افزایش سن²⁷

همانطور که در کلاس درس بیان شد، یکی از روش‌های جلوگیری از قحطی‌زدگی²⁸، استفاده از سازوکار افزایش سن است. بدین صورت که اولویت پردازه‌هایی که مدت زیادی صبر کرده‌اند و پردازنده هنوز به آن‌ها اختصاص نیافته است، به مرور افزایش می‌یابد. لذا سازوکار افزایش سن برای پردازه‌ها را به صورت زیر قرار دهید:

در زمان‌بندی بازخوردی چند سطحی‌ای که پیاده‌سازی می‌کنید، به‌جز اولین پردازه و shell که باید در صف اول قرار بگیرند، بقیه پردازه‌ها را در ابتدا به طور پیش‌فرض در صف سوم قرار دهید و در صورتی که پردازه‌ای در این صف سوم حداقل 800 سیکل منتظر مانده بود، آن را به صف دوم منتقل کنید. همچنین در صورتی که پردازه‌ای در صف دوم مجدداً حداقل 800 سیکل منتظر مانده بود، آن را به صف اول منتقل کنید.

دقت کنید که مدت زمانی که پردازه منتظر مانده است، به محض رسیدن نوبت اجرا به آن و یا فعال شدن سازوکار افزایش سن، صفر می‌شود و بعد از رها کردن پردازنده، در صورت خاتمه نیافتن آن پردازه، مجدداً به مدت انتظار آن در هر سیکل از اجرای سیستم اضافه می‌شود. در نتیجه شما باید تنها در حالتی که پردازه در وضعیت RUNNABLE قرار دارد، به مدت زمان انتظار پردازه اضافه کنید.

(۱۷) به چه علت مدت زمانی که پردازه در وضعیت SLEEPING می‌باشد به عنوان زمان انتظار پردازه از منظر زمان‌بندی در نظر گرفته نمی‌شود؟

در صورت انتقال پردازه به صف‌های دیگر، رصد کردن تعداد سیکل اجرا نشده را به صورت مداوم انجام دهید. دقت داشته باشید که با هر بار فعال شدن سازوکار افزایش سن، با استفاده از `cprintf` شماره `pid` پردازه به همراه صف مبدأ و مقصد آن را گزارش کنید.

²⁷ Aging

²⁸ Starvation

نکته: توجه کنید که پردازۀ پوسته²⁹ و همچنین پردازه‌هایی که یک fork از این پردازه می‌باشند، می‌بایست در صف اول قرار گیرند تا پوسته نسبت به دیگر پردازه‌های عادی قفل نشود. همچنین برای درستی برنامه‌های خود، می‌بایست پردازه‌هایی که از سمت پوسته exec می‌شوند را به صف پیش‌فرض انتقال دهید.

دلیل این کار به خاطر این است که پوسته برای اجرای برنامه‌ها ابتدا یک پردازۀ فرزند ایجاد می‌کند و روی آن wait را صدا می‌زند. این پردازۀ فرزند بسته به نیاز (مانند اجرای پس‌زمینه) ممکن است فرزندهای دیگری ایجاد کند که روی آن‌ها در ادامه exec انجام دهد. در نهایت، این پردازۀ فرزند exit را صدا می‌زند و پوسته از حالت wait خارج می‌شود. در صورتی که این پردازۀ فرزند در اولویت اجرا (سطح اول) نمی‌بود و به قسمت exit نمی‌رسید، پوسته در حالت wait می‌ماند و در نتیجه قفل می‌شد.

در خصوص برنامه‌هایی که در نهایت به وسیله‌ی یک exec از fork پوسته اجرا می‌شوند، اولییتی که پیش‌تر آن را بالا بردیم را به ارث می‌برند. این پردازه‌ها در مقابل پردازه‌هایی که خودشان فرزندان ایجاد می‌کنند، نسبت به این فرزندان اولویت پیدا می‌کنند و این رفتار مطلوبی نخواهد بود. برای حل این موضوع، می‌بایست در هنگام exec کردن بررسی کنیم که اگر پردازه‌ای که در حال انجام عملیات exec می‌باشد پردازۀ اولیه (initproc) یا پردازۀ پوسته (sh) نبود، صف زمان‌بندی این پردازه را به صف پیش‌فرض انتقال دهیم (برای این کار می‌توانید از فراخوانی سیستمی‌ای که برای تغییر صف می‌نویسید، استفاده کنید).

ارزیابی تجربی و مقایسه الگوریتم‌ها

در پایان، برای مشاهده و ارزیابی زمان‌بند خود و نیز مقایسه‌ی تمام الگوریتم‌های پیاده‌سازی شده، نیاز به طراحی یکسری فراخوانی‌های سیستمی³⁰ و همچنین برنامه‌های سطح کاربری داریم تا آن فراخوانی‌های سیستمی را در آن‌ها استفاده کنیم.

²⁹ Shell

³⁰ System calls

فراخوانی‌های سیستمی

فراخوانی سیستمی	توضیح
مقداردهی اولیه‌ی زمان اجرای تخمینی و سطح اطمینان	این فراخوانی سیستمی شامل سه ورودی از نوع int خواهد بود که ورودی اول pid پردازهی مورد نظر، ورودی دوم زمان اجرای تخمینی، و ورودی سوم سطح اطمینان می‌باشد.
تغییر صف ³¹	این فراخوانی سیستمی شامل دو ورودی از نوع int است که ورودی اول pid پردازهی مورد نظر و ورودی دوم نشان دهنده‌ی صف مقصد می‌باشد.
چاپ اطلاعات	با صدا زده شدن این فراخوانی سیستمی، تمامی اطلاعات مربوط به پردازه از قبیل نام، شماره pid، وضعیت، عدد صف، مدت زمان انتظار، عدد سطح اطمینان، زمان اجرای تخمینی، تعداد تیک‌هایی که پردازه پشت سر هم اجرا شده است، و زمان ورود به صف کنونی ³² چاپ می‌شوند.

در نهایت، برنامه‌ی سطح کاربری‌ای بنویسید که درون خود از این فراخوانی‌های سیستمی استفاده کرده و صحت پیاده‌سازی زمان‌بند را به تصویر می‌کشد. این برنامه می‌تواند در بخش بعدی نیز مورد استفاده قرار گیرد. توجه کنید که از sleep استفاده نکنید و به‌جایش از حلقه‌های تودرتو و محاسبات متنوع استفاده کنید تا برنامه‌ی خود را به میزان چند ثانیه در حالت قابل اجرا و یا در حال اجرا نگه دارید.

³¹ توجه داشته باشید که اگر صف مقصد نامعتبر بود و یا صف مبدا و مقصد یکسان بودند، از این اقدام جلوگیری شود. همچنین با اجرای این فراخوانی، مدت زمان منتظر ماندن پردازه نباید تغییر کند.

³² در صورت تغییر یافتن صف پردازه، زمان ورود به صف باید آپدیت شود. همچنین در هنگام ایجاد پردازه، مقدار آن برابر با زمان (ticks) سیستم است.

name	pid	state	queue	wait time	confidence	burst time	consecutive run	arrival
init	1	SLEEPING	0	0	50	2	2	0
sh	2	SLEEPING	0	0	50	2	0	0
scheduletest	3	RUNNING	2	0	50	2	3	3353
scheduletest	4	RUNNABLE	2	672	50	2	0	3354
scheduletest	5	RUNNABLE	2	672	50	2	0	3354
scheduletest	6	RUNNABLE	2	672	50	2	0	3354
scheduletest	7	RUNNABLE	2	671	50	2	0	3355
scheduletest	8	RUNNABLE	2	671	50	2	0	3355

شکل ۳. نمونه‌ای از خروجی فراخوانی سیستمی چاپ اطلاعات در ترمینال.

برنامه‌های سطح کاربری

به منظور ارزیابی عملکرد زمان‌بندهای پیاده‌سازی‌شده، ۳ برنامه‌ی سطح کاربری به صورت یک برنامه‌ی پردازش و محاسبات سبک و کوتاه، یک برنامه‌ی پردازش و محاسبات سنگین و طولانی (معادل با صبر کردن پردازش مثلاً برای عملیات I/O)، و یک برنامه‌ی ترکیبی از هردو را طراحی کرده و به کمک فراخوانی‌های سیستمی پیاده‌سازی شده، رفتار سیستم برای هر برنامه را تحت الگوریتم‌های زمان‌بندی مختلف (RR، SJF، FCFS، MFQ)، مقایسه و بررسی کنید. در نهایت، در گزارش خود توضیح دهید که کدام الگوریتم زمان‌بندی برای هر برنامه عملکرد مناسب‌تری داشته و علت آن چیست.

سایر نکات

- بخش خوبی از نمره‌ی شما را پاسخ دهی به سوالات مطرح شده تشکیل می‌دهد که به شما در درک نحوه‌ی کارکرد xv6 و پیاده‌سازی قسمت زمان‌بندی کمک می‌کند.
- پاسخ سوال‌ها را تا حد امکان مختصر و مفید بنویسید.
- در پیاده‌سازی خود، در خصوص مواردی که ذکر نشده‌اند، می‌توانید فرض منطقی و دلخواه خود را داشته باشید.
- همه افراد می‌بایست به پروژه مسلط باشند و نمره تمامی اعضای گروه لزوماً یکسان نخواهد بود.
- فصل پنجم کتاب xv6 بسیار مفید خواهد بود و مطالعه‌ی آن توصیه می‌شود.

- در صورت وجود هرگونه سوال یا ابهام می‌توانید با ایمیل دستیاران مربوطه یا گروه بله‌ی درس در ارتباط باشید.
- این تمرین صرفاً برای یادگیری شما طرح شده است. در صورت محرز شدن تقلب در تمرین، مطابق با قوانین درس برخورد خواهد شد.

موفق باشید